MōōD: Building a Hyperlocal Boutique Discovery Platform

Lauren Morgenthaler
Department of Computer Science (CMPT390) | Westminster University

The Gap

A Discovery Problem

Two in five consumers say they'd rather shop independent boutiques than chains, and 65% wish they shopped local more often — but they don't, because they can't find them. Roughly 12 million small retailers compete with Amazon and Shein, and the overwhelming majority don't show up when you search "black dress" on Google. As Brookings Institute put it: local boutiques aren't hidden because they lack style — they lack digital visibility.

MōōD is the discovery layer above their existing catalogs. Map-first, style-first, currently live across 29 independent LA boutiques and ~33K real products pulled directly from their Shopify storefrontS.

Building Like a Technical Founder

An Iterative Methodology

Shipped solo across six sprints. The technical-founder skill isn't about being right from day one— it's keeping the architecture loose enough that pivots cost days, not sprints, and paying tech debt down inside the sprint it was created. Almost every early commitment in this project got revisited:

Seeded fake data → live Shopify storefront sync. A cofounder pitching boutique owners needs something real; fake data doesn't sell.
Yelp-style freemium → phased commission + subscription. LTV:CAC came back 33:1 vs. 5:1 on the alternative; trusted the math over instinct.
Sprint 6 launch-prep → mid-sprint refactor. Round-two testing surfaced real bugs that would cost credibility on the product quality if it were to be presented in an investor pitch; paid them down in atomic redeploys before release.

The reason those pivots were cheap is the architecture was built for them — public-storefront and OAuth sync paths share one normalization pipeline, the revenue model was layered not welded, and the schema was coordinate-agnostic from day one.

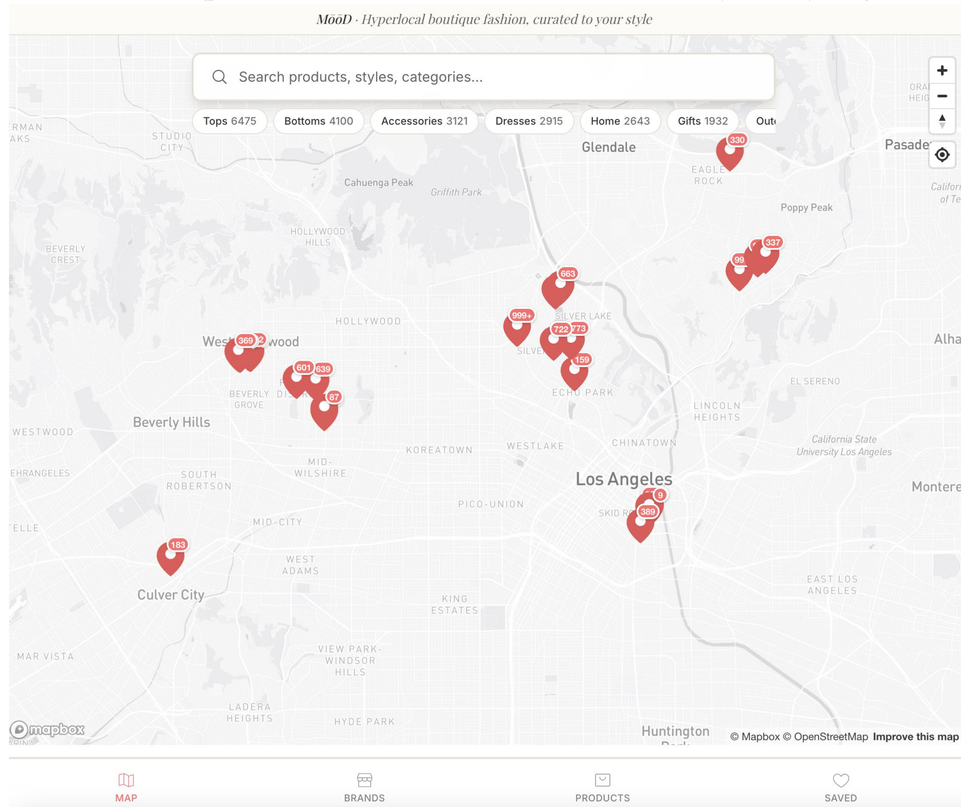
The Product

B2C App

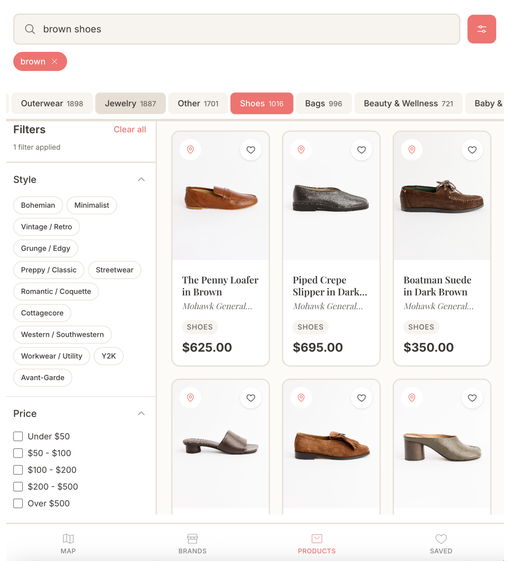
A two-sided product — think **Yelp for fashion inventory.** Consumers open the app to a map of every boutique near them, but instead of reviews, they see what's actually in stock, browsable by style ("cottagecore," "western," "bohemian") as well as category. Boutique owners get a dashboard showing who's finding them, what inventory of theirs is trending, and what their store is known. Free to list forever, commission model for once sales are driven from our site to enacted orders.

moodla.vercel.app

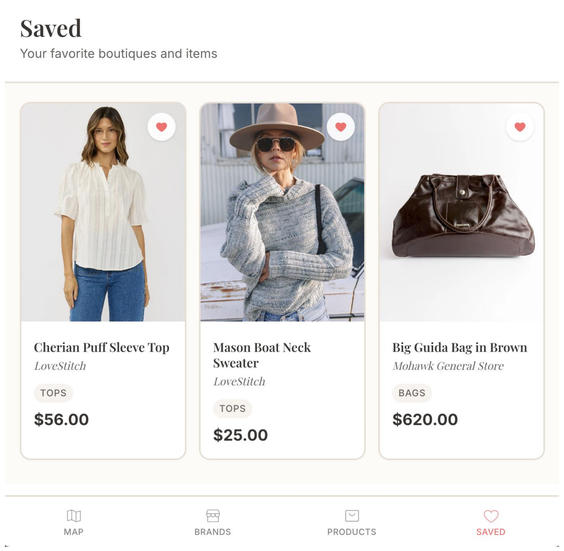
29 LA Boutiques • ~33K Products • Live Inventory, Daily Refresh



Map view — Style-first browse with server-side PostGIS filtering.

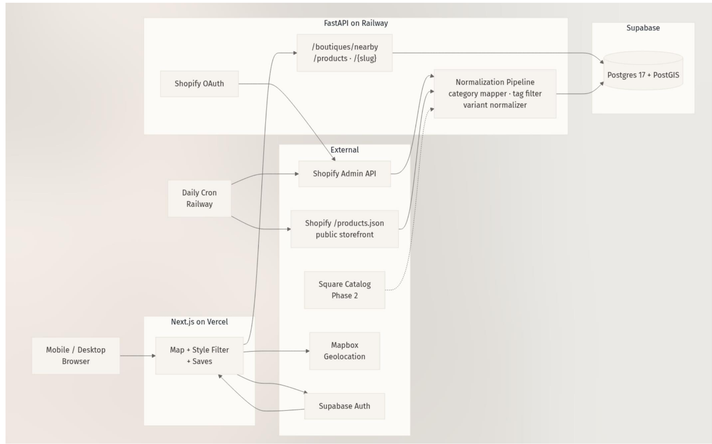


Product view – Real-time inventory pulled from each boutique's Shopify storefront, whether publicly or through OAuth access.



User Accounts/Saves — Progressive-disclosure auth: browse anonymously, sign in to save.

Architecture



The consumer flow is straightforward: the Next.js frontend sends lat/long to FastAPI, which calls a **PostGIS stored function** — spatial-indexed nearby query with product counts joined — and returns JSON in under 500ms. The interesting work happens on ingestion. **Two sync paths feed the same database**: public Shopify storefronts (/products.json, daily cron) and Shopify OAuth (Admin API, for commission-eligible partners). Both hit one normalization pipeline, so downstream querying doesn't care where a product came from.

- **Frontend** — Next.js 16, React 19, Mapbox GL, TypeScript
- **Backend** — FastAPI, Python, async SQLAlchemy + asyncpg
- **Data** — Postgres 17 + PostGIS on Supabase
- **Infra** — Railway, Vercel, UptimeRobot · ~$5/mo

Shopify OAuth (Admin API, for commission-eligible partners). Both hit one normalization pipeline, so downstream querying doesn't care where a product came from.

Learnings

01 — Shape of the data drives the system.
The string data for the product descriptors is inherently messy and mixed with promo codes, vendor names, etc. A static lookup would have broken on every new boutique, so I needed to focus on the normalization pattern and fitting a system to best match/catalog the products to descriptors in order to provide accurate filtering was one of the most crucial learnings when building this. Luckily, I can say that today ~95% of products resolve accurately and new boutiques onboard with zero manual mapping most of the time.

02 — Innovate on the product, not the process.
I kept building the "final" version of a feature before anyone had used it — I was reinventing what the MVP playbook already solved. Build the core, ship it, validate that it solves a real problem, then expand. The founder skill is knowing which layer is yours to innovate on — the product itself, the category insight — and which you inherit from the canon: how you build, validate, and sequence.